

Documentation

Software Requirement Specification (SRS)

A *Software Requirement Specification* (SRS) is a comprehensive document that details the intended purpose, environment, and behavior of software under development.

- **Core:** It *explicitly* states what the software **will do** and how it is **expected to perform**.
- **Living Reference:** It serves as the *single source of truth* for the development team, stakeholders, QA engineers, and project managers throughout the entire software development lifecycle (SDLC).

The SRS Structural Outline

A standard SRS document is broken down into the following key sections:

1. **Introduction**
2. **General Description**
3. **Functional Requirements**
4. **Non-Functional Requirements**
5. **Performance Requirements**
6. **Design Constraints**
7. **Non-Functional Attributes**
8. **Preliminary Schedule and Budget**
9. **Appendices**

1. Introduction

The introduction sets the stage for the project, aligning all stakeholders on the baseline vision.

- **Purpose:** Explains *why* this document exists and who the intended audience is.
- **Scope:** Defines the boundaries—explicitly stating what the system *will* and *will not* do.
- **Overview:** Provides a brief roadmap of how the rest of the document is structured.

Example:

This document specifies the requirements for the "Smart Pantry" mobile application, designed for household managers to track grocery inventory and reduce waste. The scope includes barcode scanning, manual item entry, and automated expiry notifications. It explicitly excludes direct online grocery ordering or third-party e-commerce integrations.

2. General Description

This section provides high-level context about the software without diving into deep technical mechanics.

- **User Objective:** What the *end-user* ultimately wants to accomplish using this software.
- **User Characteristics:** Background, expected *technical skill* level, and environmental *constraints* of the target user.
- **Features, Benefits, & Importance:** A summary of the system's core capabilities and the *value* they deliver.

Note that there would usually be multiple objectives, characteristics, and benefits in an actual project

Example:

- *User Objective:* A household manager wants to avoid buying duplicate items during grocery trips.
- *User Characteristic:* Users are expected to be familiar with basic smartphone applications but will frequently use the app while multi-tasking in a kitchen or grocery store environment.
- *Benefits:* The app provides real-time inventory visibility, saving users money and significantly reducing domestic food waste.

3. Functional Requirements

This is the core technical blueprint of the document. It **fully explains the operation** of the program.

- Every system feature *must be detailed* with clear, measurable inputs, behaviors, and outputs.
- Requirements should typically be written using objective **"shall"** statements to make them clear and testable.

Example:

When a user successfully scans a product barcode via the device camera, the system *shall* fetch the product name and category from the global database, update the user's local inventory list, and assign a default quantity of one (1).

4. Non-Functional Requirements (NFRs)

Define *how well* the system performs its functions, focusing on environmental *behavior* and *quality* attributes.

- **Performance:** System speed, response times, throughput, and resource utilization constraints.
- **Security:** Data protection, encryption, user privacy guidelines, and authentication protocols.
- **Usability:** Ease of learning, user interface efficiency, and accessibility standards (e.g., WCAG).
- **Reliability:** System availability metrics, uptime guarantees, and fault tolerance behavior.

Example:

The application *shall* synchronize local inventory changes with the cloud database within two (2) seconds of an active network connection becoming available. Furthermore, all user credentials *must* be encrypted and stored using industry-standard salted hashing algorithms.

5. Performance, Constraints, and Attributes

While closely tied to non-functional requirements, these sections impose *strict boundaries* on *engineering decisions*, architectural design, and quality metrics.

- **Performance (Quantitative):** Specific, *measurable numeric limits* (e.g., concurrent user support, database limits).
- **Design Constraints:** *Hard restrictions* imposed by technology, hardware, legacy software, or compliance (e.g., languages, frameworks, operating systems).
- **Non-Functional Attributes:** Standards for portability (cross-platform capacity), maintainability (code readability), and testability.

Example:

The system *must* be developed using the React Native framework to ensure cross-platform compatibility for both iOS (version 15+) and Android from a single codebase.

6. Schedule, Budget, and Appendices

The practical operational details

- **Schedule:** Major project milestones, sprint breakdowns, and hard delivery deadlines.
- **Budget:** Estimated financial costs spanning development hours, cloud hosting architectures, and third-party API dependencies.
- **Appendices:** A glossary of technical terms, system architecture diagrams, UX wireframes, and external reference documentation.

Example:

- **Milestone 1:** Initial UI Prototype & Wireframes delivery (Due: June 15).
- **Glossary:** "*Inventory Sync*" refers to the automated, bi-directional reconciliation process between the local device storage state and the remote cloud database state.

Examples

Note that these do not follow the exact structure outlined above, but they do provide a good reference for the level of detail and clarity expected in an SRS document.

- <https://www.enabel.be/app/uploads/2025/06/Annex-A-Detailed-Software-Requirements-Specification-SRS.pdf>
- <https://www.cse.iitk.ac.in/users/pankajjalote/OldSEbookSite/CaseStudies/CaseStudy1/SRS.pdf>
- https://dspmuranchi.ac.in/pdf/Blog/srs_template-ieee.pdf
- https://dspmuranchi.ac.in/pdf/Blog/srs_template-ieee.pdf

Activity: SRS Document Creation

Working with your established project groups, collaborate to build a comprehensive SRS document for your assigned software project. Ensure you follow the structure detailed above.

Work in Google Docs and include a URL to the document. Ensure that all group members have edit permissions to the document.

This is so that I can check the edit history for individual grading

Note that the specific content of the SRS does not have to be consistent, it just has to be comprehensive enough to understand the project and the requirements.
The goal is to practice writing clear, detailed, and testable requirements.

Submission Guidelines

- *File Format:* PDF format only (.pdf)
- *File Naming Convention:* `groupname_srs.pdf`
- *Submission Platform:* **Each** member must upload the final document on *NEO*.

120 minutes

Business Scenarios for Assignment

Choose one of the following scenarios for your SRS document:

1. **Music School:** A lesson scheduling system for students and teachers, including payment processing and progress tracking.
2. **Community Garden:** A plot reservation system with water-usage tracking and shared tool inventory management.
3. **Youth Sports League:** A team registration portal with game schedule generation and automated referee assignments.
4. **Pet Grooming Salon:** An online booking platform with service-level selection and automated appointment reminders.
5. **Artisan Craft Marketplace:** An inventory tracker for small-batch creators, supporting event pop-up logistics and vendor booth mapping.
6. **Neighborhood Tool Library:** A lending system for specialized home tools, including membership management and equipment condition reporting.

Business Scenarios for Assignment

7. **Bicycle Repair Shop:** A maintenance tracking portal where customers can check the real-time status of their repairs and receive quote approvals.
8. **Local Theater Group:** A ticket sales system with seat selection mapping, volunteer sign-up sheets, and show-night cast notifications.
9. **Urban Farm Share:** A subscription box management system for local vegetable shares, including harvest updates and custom delivery scheduling.
10. **Dance Studio:** An instructor portal for managing student class levels, recital choreography documentation, and costume measurement tracking.
11. **Mobile Car Wash:** An on-demand service app where users pin their vehicle location, select detailing packages, and receive status updates during the wash.
12. **Residential Property Manager:** A maintenance request portal for tenants, integrating automated vendor dispatching and cost tracking for building owners.